DBMS Project — Demo Sheet

Complete before the lab demo. Submit as a single-page PDF. List source-file paths and line numbers for queries/features.

Team ID / Lab section	Members (names and IDs)	GitHub repository / final commit hash
Group 17 (CineBook: Movie Ticket Booking System) / P2	Harinandha L 2024A4PS0660H; Imran Hafeez S 2024A4PS0690H; Arjun Rathore 2024A4PS0691H; Sushanth Reddy Kompally 2024A8PS0459H; Pratham Singla 2024A8PS0486H	github.com/Pratham15-svg/CineBook, tag `final`, commit `2389733`
RDBMS and technology stack	**Number of tables**	**Total database records**
PostgreSQL 16; Node.js/Express REST API (node-postgres, JWT, bcrypt); React + Vite; Docker Compose	13 (`app_user, city, venue, screen, seat, movie, movie_show, show_price, coupon, booking, booking_seat, payment, booking_audit`)	10,044 seeded rows (`db/seed/generate_seed_data.py`): 988 bookings, 2,728 booked seats, 1,970 seats, 300 shows

A. Application workflows (at least 3)

#	Workflow / purpose	DBMS aspect	Tables involved	UI action → database effect
1	**Customer books tickets** (Shows → Seat selection → Checkout → My Bookings); optional coupon; cancel with refund	Transaction in stored procedures `sp_create_booking`, `sp_confirm_booking`, `sp_cancel_booking`; partial unique index (no double booking); deferred constraint triggers; audit trigger	booking, booking_seat, seat, show_price, coupon, payment, booking_audit	*Hold seats* → INSERT booking (HELD, 10-min expiry) + booking_seat with `list_price` snapshot (`bookingController.js:48`). *Pay* → payment SUCCESS, booking CONFIRMED, `price_paid` set (:100). *Cancel* → CANCELLED, seats freed (`is_active=false`), refund recorded (:141).
2	**Admin analytics dashboard** (revenue per venue per month, occupancy per show, top movies per city, peak hours)	Complex retrieval through views: multi-table joins, GROUP BY, window function, CTE, LATERAL subqueries	booking, booking_seat, movie_show, movie, screen, venue, city, payment	*Open Admin Dashboard* → four read-only `SELECT * FROM v_…` queries (`adminController.js:391, 406, 421, 437`); no data changed.
3	**Manager schedules a show; invalid operations rejected** (Schedule Show page)	Transaction (BEGIN/COMMIT/ROLLBACK); exclusion constraint (no overlapping shows); CHECK constraints; role trigger	movie_show, show_price, screen, venue, movie, app_user	*Schedule Show* → INSERT movie_show + one show_price row per seat category in one transaction (`managerController.js:197-313`). Same screen and time again → `excl_show_no_overlap` violation → HTTP 409. Test log: `db/tests/w3_tests.sql` (38 tests).

B. Non-trivial SQL queries (at least 5)

#	Query purpose	SQL concepts (join, group, subquery, CTE, etc.)	Source file : line	Workflow / screen
1	Show search: price range, total and taken seats per show, with filters	5-table join, three `LEFT JOIN LATERAL` aggregate subqueries (MIN/MAX, COUNT)	`db/05_indexes_views.sql:128` (`v_show_search`); used at `showController.js:19`	W1, Shows page (search and filters)
2	Revenue per venue per month (net of refunds)	Multi-table join, GROUP BY, `date_trunc`, SUM	`db/05_indexes_views.sql:247` (`v_revenue_venue_month`); `adminController.js:391`	W2, Admin Dashboard
3	Seat occupancy per show	Join, LATERAL subqueries, COUNT … FILTER, ratio	`db/05_indexes_views.sql:270` (`v_show_occupancy`); `adminController.js:406`	W2, Admin and Manager dashboards
4	Top 3 movies per city by tickets sold	CTE (`WITH sales AS`), `ROW_NUMBER() OVER (PARTITION BY city)`	`db/05_indexes_views.sql:302` (`v_top_movies_city`); `adminController.js:421`	W2, Admin Dashboard
5	Peak booking hours	GROUP BY on `EXTRACT(HOUR …)` with time-zone conversion, COUNT	`db/05_indexes_views.sql:333` (`v_peak_hours`); `adminController.js:437`	W2, Admin Dashboard

C. Additional DBMS features (at least 3)

#	Feature (transaction / view / trigger / procedure / index)	Object / function name	Source file : line	Purpose / effect
1	Stored procedure + transaction	`sp_create_booking`, `sp_confirm_booking`, `sp_cancel_booking`	`db/04_procedures.sql:13, 162, 247`	Hold, confirm and cancel a booking atomically; coupon validated under `FOR UPDATE`; 2-hour cancel cutoff and 100%/50% refund.
2	Triggers (6 in total)	`trg_booking_audit`, `trg_release_seats`, `trg_booking_seat_count`, `trg_booking_confirmation_check`, `trg_venue_manager_role`, `trg_booking_show_in_future`	`db/03_triggers.sql:56, 82, 118, 191, 31`; `db/05_indexes_views.sql:50`	Audit every status change; free seats on cancel/expiry; 1 to 6 seats and a balanced payment checked at commit; manager must have role MANAGER; no booking for a past show.
3	Index (partial unique)	`uq_active_seat_per_show`	`db/01_tables.sql:187`	Only one active booking per seat per show: a double booking fails with a unique violation.
4	Exclusion constraint (needs `btree_gist`)	`excl_show_no_overlap`	`db/02_constraints.sql:41`	Two shows on one screen cannot overlap in time; back-to-back allowed.
5	Function + view + other indexes	`fn_seat_map`, `v_ticket`; 12 indexes	`db/05_indexes_views.sql:176, 209, 14-28`; `db/01_tables.sql:141`	Seat map with status and price; ticket from the stored snapshot (one lateral aggregate, no joins to user, show or movie); speeds up search and my-bookings.